UNIVERSIDADE FEDERAL DE ALAGOAS (UFAL)

INSTITUTO DE COMPUTAÇÃO / ENGENHARIA DE COMPUTAÇÃO

Lista de Exercícios 2: Inteligência Artificial

Componente Curricular: ECOM031 (Graduação / PPGI)

Bruno Gomes de Barros Filho, matrícula 21111360

Antonio Rodrigo Lima de Andrade Tenório, matrícula 23210707

Professor: Evandro de Barros Costa

Maceió, AL, 2026.2

Sumário

1 Questão 1: Representação do Conhecimento: Ontologias versus Grafos de Conhecimento **3**

1.1 Comparativo entre sistemas baseados em ontologias e grafos de conhecimento . . . 3

1.1.1 Fundamentação teórica, semântica e suposição de mundo . . . 3

1.1.2 Mecanismos de raciocínio e inferência . . . 3

1.1.3 Escalabilidade e complexidade computacional . . . 4

1.2 Exemplo de aplicação: catálogo e recomendação cinematográfica . . . 4

1.2.1 Abordagem por ontologia (modelagem terminológica: TBox) . . . 4

1.2.2 Abordagem por grafo de conhecimento (modelagem estrutural: LPG) . . . 5

1.3 Mapeamento de uma ontologia para um grafo de conhecimento . . . 5

2 Questão 2: Classificador dos K-Vizinhos Mais Próximos (KNN) **7**

2.1 Base de dados e configuração do experimento . . . 7

2.2 As 30 instâncias utilizadas no experimento . . . 7

2.3 Funcionamento do algoritmo KNN . . . 8

2.4 Seleção do hiperparâmetro k via Leave-One-Out (LOO-CV) . . . 8

2.5 Resultados e análise de desempenho . . . 9

3 Questão 3: Chatbot ELIZA, diálogo, algoritmo e evoluções **11**
3.1 O chatbot ELIZA e o script DOCTOR 11
3.2 Trecho de diálogo e análise técnica 11
3.3 Regras e algoritmo do ELIZA 12
3.4 Resposta do ELIZA a cada sentença do diálogo 12
3.5 Evoluções: PARRY (1972) e AIML / ALICE (1995–2000) 13

1 Questão 1: Representação do Conhecimento: Ontologias versus Grafos de Conhecimento

1.1 Comparativo entre sistemas baseados em ontologias e grafos de conhecimento

Na Representação do Conhecimento e Raciocínio (RCR), a modelagem formal de domínios veio das redes semânticas e dos sistemas de *frames* das décadas de 1970 e 1980 e chegou a duas linhas muito usadas hoje: ontologias formais e grafos de conhecimento (*Knowledge Graphs*). Russell e Norvig (2013) lembram que qualquer formalismo precisa equilibrar a expressividade necessária à inferência dedutiva com a tratabilidade dos algoritmos.

As duas abordagens organizam entidades e relacionamentos em estruturas direcionadas, mas partem de fundamentos e objetivos distintos.

1.1.1 Fundamentação teórica, semântica e suposição de mundo

As ontologias formais se apoiam nas Lógicas de Descrição (*Description Logics*, DL), fragmentos decidíveis da Lógica de Primeira Ordem. Esse arcabouço deu origem ao padrão W3C OWL 2 (*Web Ontology Language*), com semântica baseada na lógica $\mathcal{SROIQ}$. Classes, relações e instâncias são descritas por axiomas, sob a Hipótese de Mundo Aberto (*Open World Assumption*, OWA): a ausência de uma afirmação na base não autoriza concluir que ela é falsa; só indica que a informação é desconhecida. Também se adota a ausência da pressuposição de nomes únicos (*Non-Unique Name Assumption*): indivíduos com identificadores distintos podem ser inferidos como equivalentes (`owl:sameAs`), salvo declaração explícita de desigualdade.

Os grafos de conhecimento surgiram da convergência entre a Web Semântica (RDF e RDF Schema) e bancos orientados a grafos, em especial o modelo de Grafos de Propriedades Rotuladas (*Labeled Property Graphs*, LPG; por exemplo, Neo4j). A prioridade é a representação estruturada e a conectividade em larga escala de instâncias reais. Nas consultas LPG, costuma-se adotar a Hipótese de Mundo Fechado (*Closed World Assumption*, CWA) com pressuposição de nomes únicos: a ausência de uma aresta entre dois nós é tratada como inexistência do vínculo, o que facilita avaliações de conectividade.

1.1.2 Mecanismos de raciocínio e inferência

Nos sistemas ontológicos, o raciocínio é dedutivo. Motores como HermiT, Pellet e FaCT++, baseados em algoritmos de *Tableau*, processam axiomas terminológicos (TBox) e assertivos (ABox). As tarefas usuais incluem:

- verificação de consistência: a ontologia admite pelo menos um modelo satisfatível?
- subsunção e classificação taxonômica: calcular a hierarquia implícita de classes a partir das restrições declaradas;
- instanciação e realização: decidir a quais classes complexas um indivíduo pertence.

Nos grafos de conhecimento, a inferência principal deixa de ser a dedução axiomática e passa a ser estrutural, algorítmica ou estatística. Usam-se consultas por padrão de

subgrafo (Cypher ou SPARQL), algoritmos de teoria dos grafos (caminhos mínimos de Dijkstra, centralidade de PageRank, coeficientes de agrupamento) e aprendizado sobre grafos, como embeddings de entidades e relações (*Knowledge Graph Embeddings*; TransE, RotatE) e redes neurais em grafo (GNNs). O objetivo típico é predizer elos ausentes (*link prediction*) e alinhar entidades em bases ruidosas.

1.1.3 Escalabilidade e complexidade computacional

O poder expressivo das Lógicas de Descrição tem custo alto. Em OWL 2 DL, a satisfatibilidade no pior caso chega a ordens de NEXPTIME a 2EXPTIME, o que limita raciocinadores completos sobre bases com dezenas de milhões de indivíduos.

Bancos de grafo orientados a propriedades foram pensados para leitura, escrita e travessia em grafos com bilhões de nós e arestas. Ao abrir mão da checagem automática de axiomas complexos em tempo de execução, usam indexação direta por ponteiros (*index-free adjacency*): o custo de travessia local depende do grau dos nós envolvidos, e não do tamanho global do grafo.

A Tabela 1 resume os pontos de diferença.

Tabela 1: Síntese comparativa entre Sistemas Baseados em Ontologias e Grafos de Conhecimento

Critério de Análise	**Sistemas Baseados em Ontologias**	**Grafos de Conhecimento (LPG/RDF)**
Fundamentação Teórica	Lógicas de Descrição ($\mathcal{SROIQ}$), Lógica de Primeira Ordem	Teoria dos Grafos, Bancos de Grafos de Propriedades
Especificações Comuns	W3C OWL 2, RDF Schema, Protégé	Labeled Property Graph (Neo4j), RDF Triplestores
Suposição de Mundo	Mundo Aberto (OWA), sem nomes únicos obrigatórios	Mundo Fechado (CWA) em consultas, nomes únicos explícitos
Natureza da Inferência	Dedução lógica formal via Tableau (HermiT, Pellet)	Travessia de padrões (Cypher), centralidade, embeddings, GNNs
Foco Funcional	Validação axiomática, consistência conceitual, subsunção	Consultas de conectividade, travessia rápida, escalabilidade
Complexidade de Escala	Elevada complexidade de raciocínio (NEXPTIME / 2EXPTIME)	Travessia em tempo quase linear por adjacência direta

1.2 Exemplo de aplicação: catálogo e recomendação cinematográfica

Para contrastar as duas formas de representação, considere um catálogo com recomendação de filmes.

1.2.1 Abordagem por ontologia (modelagem terminológica: TBox)

Na abordagem ontológica, o objetivo é formalizar o esquema conceitual e as regras de integridade do domínio:

- Taxonomia de classes: definem-se `Pessoa`, `Diretor`, `Ator` e `Filme`, com `Diretor` $\sqsubseteq$ `Pessoa`, `Ator` $\sqsubseteq$ `Pessoa` e o axioma de disjunção `Pessoa` $\sqcap$ `Filme` $\equiv \bot$.
- Propriedades de objeto: modela-se a relação binária `dirigiu`, com domínio `Diretor` ($\exists$`dirigiu`.$\top \sqsubseteq$ `Diretor`) e imagem `Filme` ($\top \sqsubseteq \forall$`dirigiu`.`Filme`).
- Raciocínio automático: ao inserir na ABox o fato `dirigiu(Christopher_Nolan, Oppenheimer)`, o raciocinador conclui que `Christopher_Nolan` pertence a `Diretor` e, portanto, a `Pessoa`, sem anotação manual. Se um registro inconsistente declarar que um `Filme` dirigiu outro `Filme`, o motor detecta a violação de domínio e aponta a inconsistência da base.

1.2.2 Abordagem por grafo de conhecimento (modelagem estrutural: LPG)

No grafo, o foco é a conectividade eficiente sobre o conjunto instanciado:

- Estrutura de nós: entidades reais viram nós com rótulos e propriedades literais, como `(:Pessoa:Diretor {nome: "Steven Spielberg"})` e `(:Filme {titulo: "Jurassic Park", ano: 1993})`.
- Arestas tipadas: os vínculos são arestas direcionadas explícitas, por exemplo `[:DIRIGIU]`, `[:ATUOU_EM]` e `[:PERTENCE_AO_GENERO]`.
- Consulta: para recomendar a partir de `"Jurassic Park"`, o sistema executa uma consulta Cypher que recupera filmes com nós adjacentes de direção, elenco ou gênero em até dois saltos:

```
MATCH (f1:Filme {titulo: "Jurassic Park"})<-[:DIRIGIU]-(d:
    Diretor)-[:DIRIGIU]->(f2:Filme)
WHERE f1 <> f2
RETURN f2.titulo, count(*) AS peso ORDER BY peso DESC
    LIMIT 5;
```

  A resposta sai em poucos milissegundos pela navegação nos índices do grafo, sem checagem dedutiva de modelos lógicos.

1.3 Mapeamento de uma ontologia para um grafo de conhecimento

Passar de uma especificação OWL 2 para um grafo de propriedades rotuladas (LPG) exige um conjunto de transformações estruturais:

1. Classes ontológicas para rótulos de nós (*labels*): cada classe OWL vira um rótulo aplicável aos nós. A herança (`rdfs:subClassOf`) pode ser tratada de duas formas: (i) vários rótulos no mesmo nó, de modo que uma instância de `Diretor` receba `:Pessoa` e `:Diretor`; ou (ii) nós de metadados para as classes, ligados por arestas `[:SUBCLASSE_DE]`.
2. Propriedades de objeto para arestas direcionadas tipadas: as *Object Properties* de OWL, relações binárias entre indivíduos, viram arestas tipadas no LPG. A propriedade `dirigiu` (domínio `Diretor`, imagem `Filme`) materializa-se como `[:DIRIGIU]` do nó de origem ao de destino.

3. Propriedades de tipo de dados para atributos chave-valor: as *Datatype Properties* mapeiam indivíduos a literais (`xsd:string`, `xsd:integer`, `xsd:dateTime`). No LPG, esses pares não precisam ser nós isolados: ficam como propriedades internas no nó correspondente (por exemplo, `anoLancamento: 1993` no nó do filme).

4. Axiomas e restrições para integridade e camada de aplicação: bancos de grafo comuns não avaliam axiomas lógicos em tempo de execução. Cardinalidade e unicidade (`owl:hasKey`) viram restrições de esquema (ex.: `CREATE CONSTRAINT FOR (f:Filme) REQUIRE f.id IS UNIQUE`). Regras mais complexas, como disjunções e propriedades transitivas, ficam na camada de serviço antes da persistência.

A Figura 1 ilustra o mapeamento.

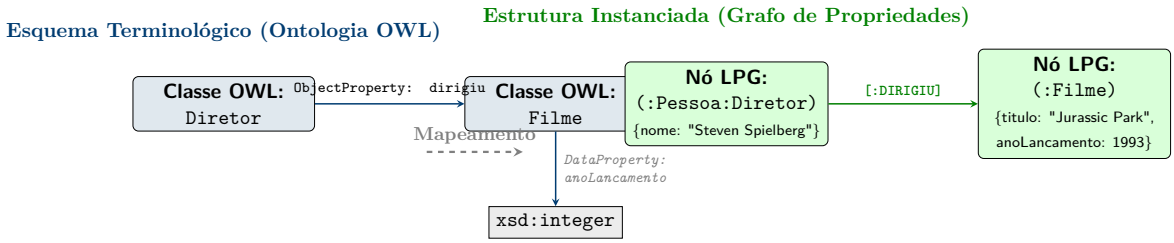


Figura 1: Mapeamento do modelo ontológico formal para grafo de propriedades rotuladas.

2 Questão 2: Classificador dos K-Vizinhos Mais Próximos (KNN)

2.1 Base de dados e configuração do experimento

O algoritmo dos K-Vizinhos Mais Próximos (*K-Nearest Neighbors*, KNN) foi avaliado sobre as mesmas 30 instâncias de risco de crédito da Lista de Exercícios 1. A base parte do exemplo clássico do livro de Luger e foi ampliada na Questão 1 daquela lista para 6 atributos categóricos e 30 exemplos (E1 a E30).

Os atributos preditores são História de Crédito, Dívida, Garantia, Renda, Emprego e Imóvel. A variável-alvo é o risco de crédito, com três classes: Alto, Moderado e Baixo.

Como o KNN usa distância numérica e todos os atributos são categóricos, cada atributo foi convertido em colunas binárias por *one-hot encoding* (`get_dummies`), a mesma estratégia do CART na Lista 1. O espaço resultante tem 14 dimensões com valores em $\{0, 1\}$, e a distância euclidiana entre dois exemplos conta, na prática, quantas categorias diferem. Não foi necessária normalização min-max.

2.2 As 30 instâncias utilizadas no experimento

A Tabela 2 reproduz a base completa da Lista 1.

Tabela 2: As 30 instâncias de risco de crédito da Lista 1

Ex.	História	Dívida	Garantia	Renda	Emprego	Imóvel	Risco
E1	Ruim	Alta	Nenhuma	0–15	Instável	Alugado	Alto
E2	Desconhecida	Alta	Nenhuma	15–35	Instável	Alugado	Alto
E3	Desconhecida	Baixa	Nenhuma	15–35	Estável	Alugado	Moderado
E4	Desconhecida	Baixa	Nenhuma	0–15	Instável	Alugado	Alto
E5	Desconhecida	Baixa	Nenhuma	>35	Estável	Próprio	Baixo
E6	Desconhecida	Baixa	Adequada	>35	Estável	Próprio	Baixo
E7	Ruim	Baixa	Nenhuma	0–15	Instável	Alugado	Alto
E8	Ruim	Baixa	Adequada	>35	Estável	Alugado	Moderado
E9	Boa	Baixa	Nenhuma	>35	Estável	Próprio	Baixo
E10	Boa	Alta	Adequada	>35	Estável	Próprio	Baixo
E11	Boa	Alta	Nenhuma	0–15	Instável	Alugado	Alto
E12	Boa	Alta	Nenhuma	15–35	Estável	Alugado	Moderado
E13	Boa	Alta	Nenhuma	>35	Estável	Alugado	Baixo
E14	Ruim	Alta	Nenhuma	15–35	Instável	Alugado	Alto
E15	Boa	Baixa	Adequada	15–35	Estável	Próprio	Baixo
E16	Boa	Baixa	Nenhuma	15–35	Estável	Próprio	Baixo
E17	Desconhecida	Baixa	Adequada	>35	Instável	Próprio	Moderado
E18	Ruim	Alta	Nenhuma	>35	Estável	Alugado	Moderado
E19	Desconhecida	Alta	Nenhuma	0–15	Estável	Alugado	Alto
E20	Boa	Baixa	Adequada	>35	Estável	Alugado	Baixo
E21	Ruim	Baixa	Nenhuma	15–35	Estável	Alugado	Moderado
E22	Boa	Alta	Adequada	15–35	Estável	Próprio	Moderado
E23	Ruim	Alta	Adequada	0–15	Instável	Próprio	Alto
E24	Desconhecida	Baixa	Nenhuma	15–35	Instável	Próprio	Moderado
E25	Boa	Baixa	Nenhuma	0–15	Estável	Próprio	Moderado
E26	Ruim	Baixa	Nenhuma	>35	Instável	Alugado	Alto
E27	Desconhecida	Alta	Adequada	>35	Estável	Próprio	Baixo
E28	Ruim	Alta	Nenhuma	15–35	Estável	Alugado	Alto
E29	Boa	Baixa	Nenhuma	>35	Instável	Próprio	Baixo
E30	Desconhecida	Baixa	Nenhuma	0–15	Estável	Alugado	Alto

A distribuição ficou com 11 exemplos de risco Alto, 9 de Moderado e 10 de Baixo.

2.3 Funcionamento do algoritmo KNN

O KNN é um algoritmo de aprendizado baseado em instâncias (*lazy learner*): não constrói uma função de generalização explícita no treino. O processamento acontece na predição:

1. Cálculo de distância: para uma instância de teste $\mathbf{q} \in \mathbb{R}^p$, calcula-se a distância euclidiana a cada ponto $\mathbf{x}_i$ do treino:

$$d(\mathbf{q}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{p} (q_j - x_{ij})^2} \tag{1}$$

2. Seleção dos k vizinhos: ordenam-se as distâncias em ordem crescente e ficam os k pontos mais próximos:

$$\mathcal{N}_k(\mathbf{q}) = \arg\min_{S \subset D, |S|=k} \sum_{\mathbf{x}_i \in S} d(\mathbf{q}, \mathbf{x}_i) \tag{2}$$

3. Voto majoritário: a classe predita $\hat{y}$ é a mais frequente entre os k vizinhos:

$$\hat{y} = \arg\max_{c \in C} \sum_{\mathbf{x}_i \in \mathcal{N}_k(\mathbf{q})} \mathbb{I}(y_i = c) \tag{3}$$

   onde $\mathbb{I}(\cdot)$ é a função indicadora. Em empate, usam-se critérios determinísticos (menor distância média ou classe majoritária a priori).

2.4 Seleção do hiperparâmetro k via Leave-One-Out (LOO-CV)

Com amostra pequena ($N = 30$), a validação cruzada *Leave-One-Out* (LOO-CV) é adequada para escolher k: cada uma das 30 instâncias entra uma vez como teste e as outras 29 treinam o modelo.

Foram avaliados valores ímpares $k \in \{1, 3, 5, 7, 9\}$ para reduzir empates. A Tabela 3 e a Figura 2 mostram os resultados.

Tabela 3: Desempenho da Validação Cruzada Leave-One-Out por valor de k

Valor de k	**Acertos (/30)**	**Acurácia LOO-CV (%)**
$k = 1$	18	60,00%
$k = 3$	19	63,33%
$\mathbf{k = 5}$	**21**	**70,00%**
$k = 7$	19	63,33%
$k = 9$	18	60,00%

O melhor valor foi $k = 5$, com 21 acertos em 30 (70,00%). Com $k = 1$ o classificador fica sensível a vizinhos isolados; com $k \geq 7$ a votação começa a misturar exemplos de classes vizinhas e a acurácia cai de novo.

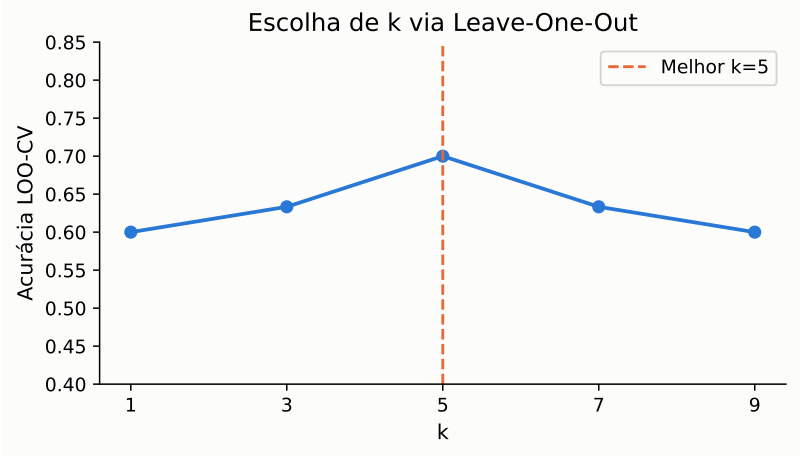


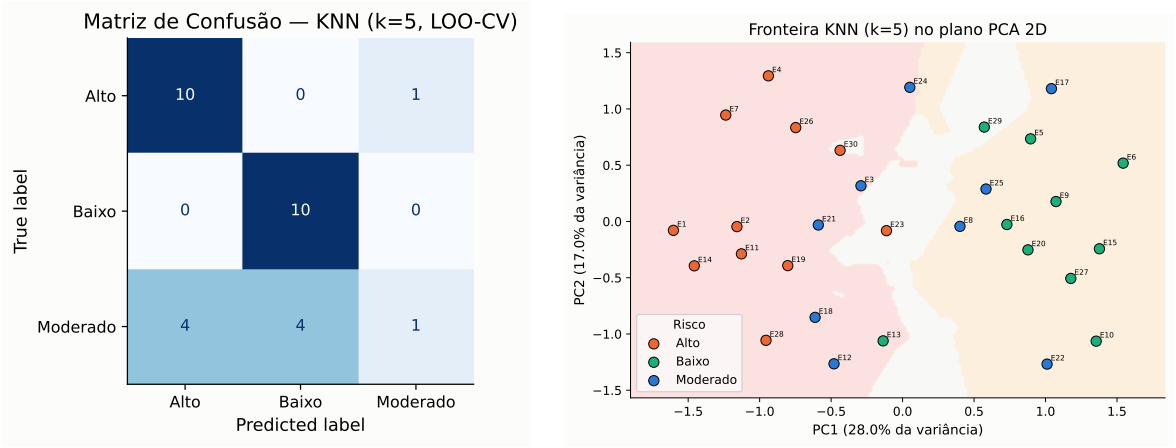
Figura 2: Curva de acurácia da validação cruzada Leave-One-Out (LOO-CV) em função de k.

2.5 Resultados e análise de desempenho

Com $k = 5$, as predições Leave-One-Out geram a matriz de confusão e as métricas da Tabela 4 e da Figura 3a.

Tabela 4: Métricas LOO-CV do KNN com $k = 5$ sobre as 30 instâncias

Classe	**Precisão**	**Revocação**	F_1**-Score**	**Suporte**
Alto	0,71	0,91	0,80	11
Baixo	0,71	1,00	0,83	10
Moderado	0,50	0,11	0,18	9
Acurácia Global		**70,00%**		**30**
Média Macro	0,64	0,67	0,61	30
Média Ponderada	0,65	0,70	0,63	30



(a) Matriz de Confusão ($k = 5$)

(b) Fronteira projetada em PCA 2D

Figura 3: Visualizações do KNN ($k = 5$): matriz de confusão em LOO-CV e fronteira no plano das duas primeiras componentes principais.

O classificador acerta bem as classes extremas: recupera 10 dos 11 exemplos `Alto` e todos os 10 `Baixo`. A dificuldade fica em `Moderado`, que no Leave-One-Out é confundida com Alto ou Baixo (apenas 1 acerto em 9). Isso faz sentido na base: o risco moderado ocupa uma região intermediária do espaço de atributos e, com poucos exemplos, os vizinhos tendem a puxar o voto para uma das extremidades.

Como exemplo passo a passo, o caso E1 (risco real Alto) tem como cinco vizinhos mais próximos E7, E11, E14, E2 e E4, todos da classe Alto. O voto é unânime e a predição coincide com o rótulo verdadeiro.

3 Questão 3: Chatbot ELIZA, diálogo, algoritmo e evoluções

3.1 O chatbot ELIZA e o script DOCTOR

O ELIZA foi desenvolvido entre 1964 e 1966 por Joseph Weizenbaum no MIT Artificial Intelligence Laboratory [5]. O programa conversa por casamento de padrões e transformação de texto. Weizenbaum não queria modelar compreensão semântica; queria mostrar que regras simples bastavam para o interlocutor projetar inteligência e empatia no sistema (o *efeito ELIZA*).

O script mais famoso é o DOCTOR, que emula um psicoterapeuta rogeriano (abordagem centrada na pessoa, de Carl Rogers). Nesse estilo, o terapeuta evita opinar, diagnosticar ou trazer temas externos: reformula o que o paciente disse e devolve a fala em pergunta ou comentário reflexivo. O ELIZA se aproveita desse domínio reduzido.

Principais funcionalidades.

1. Casamento de padrões: a sentença é decomposta a partir de palavras-chave e curingas.
2. Remontagem: trechos capturados entram em moldes de resposta, com troca de pessoa gramatical (`I/me/my` → `you/your`).
3. Pilha de palavras-chave (*keystack*): cada palavra-chave tem um peso; a de maior prioridade é processada primeiro.
4. Memória de tópicos: algumas declarações (em especial com `my`) são guardadas para reaparecer se a conversa estagnar.
5. Respostas de contingência: sem palavra-chave reconhecida, o sistema usa frases neutras (*"Please go on"*, *"What does that suggest to you?"*).

3.2 Trecho de diálogo e análise técnica

O diálogo abaixo adapta, em português, a interação clássica de Weizenbaum (1966) com o script DOCTOR.

```
[U1] Homens são todos iguais.
[E1] DE QUE MANEIRA?
[U2] Eles estão sempre nos incomodando por causa de alguma coisa.
[E2] VOCÊ CONSEGUE PENSAR EM UM EXEMPLO ESPECÍFICO?
[U3] Bem, meu namorado me fez vir aqui.
[E3] SEU NAMORADO FEZ VOCÊ VIR AQUI?
[U4] Ele disse que eu fico deprimida quase o tempo todo.
[E4] SINTO MUITO EM OUVIR QUE VOCÊ FICA DEPRIMIDA.
[U5] É verdade. Eu sou infeliz.
[E5] VOCÊ ACHA QUE VIR AQUI VAI TE AJUDAR A NÃO SER INFELIZ?
[U6] Eu preciso de um pouco de ajuda, isso é certo.
[E6] O QUE SIGNIFICARIA PARA VOCÊ SE VOCÊ CONSEGUISSE AJUDA?
[U7] Talvez eu pudesse aprender a me dar bem com minha mãe.
[E7] FALE-ME MAIS SOBRE A SUA FAMÍLIA.
[U8] Você não gosta de mim.
[E8] O QUE FAZ VOCÊ PENSAR QUE EU NÃO GOSTO DE VOCÊ?
[U9] Você não é uma pessoa de verdade, você é um computador.
```

```
[E9] IMPORTA SE EU SOU UM COMPUTADOR?
```

Partes tecnicamente interessantes. Em [U3]→[E3], a palavra-chave ligada a `meu/my` captura o segmento possessivo; a troca pronominal e a remontagem interrogativa geram *"Seu namorado fez você vir aqui?"*. Em [U4]→[E4], o estado *deprimida* dispara um molde de empatia. Em [U7]→[E7], `mãe` é generalizada para o tópico *família*, em vez de ecoar a frase inteira. Em [U2]→[E2], `sempre/always` pede um exemplo concreto, no estilo rogeriano.

Partes tecnicamente falhas. Não há memória semântica entre turnos: cada sentença é quase isolada. Negações e escopo lógico confundem o casamento ("não estou infeliz" ainda pode acionar a regra de *infeliz*). Em [U9], a exposição do truque ("você é um computador") só recebe um molde de negação de pressuposto. Em [E5], a pergunta sobre "vir aqui" presume continuidade terapêutica que [U5] não afirmou: o molde veio da palavra-chave, não de um plano de diálogo.

3.3 Regras e algoritmo do ELIZA

O motor é determinístico e opera em fases: (1) normalização e tokenização; (2) varredura de palavras-chave e montagem da *keystack*; (3) casamento com regras de decomposição; (4) substituição pronominal; (5) remontagem no próximo molde da lista circular. Se nenhuma regra da palavra-chave atual casar, o algoritmo desce na pilha; se a pilha acabar, usa memória de tópicos ou *fallback*.

Exemplo clássico para a palavra-chave `I REMEMBER`: o padrão `(* I REMEMBER *)` decompõe *"Well, I remember when we lived in Maceió"* em três partes; o molde `(DO YOU OFTEN THINK OF (3) ?)` produz *"DO YOU OFTEN THINK OF WHEN YOU LIVED IN MACEIO?"*, depois da troca pronominal nos segmentos capturados.

3.4 Resposta do ELIZA a cada sentença do diálogo

A Tabela 5 liga cada turno do usuário à palavra-chave dominante, à transformação e à saída (versão em português do trecho acima). Em todos os casos o "entendimento" é ilusório: o sistema ranqueia palavras-chave, decompõe, troca pronomes e preenche um molde.

Tabela 5: Mapeamento turno a turno do diálogo com o script DOCTOR.

Turno	**Gatilho**	**Transformação**	**Saída**
U1→E1	generalização	molde interrogativo fixo	De que maneira?
U2→E2	`sempre/always`	pede concretude	Exemplo específico?
U3→E3	`meu/my`	captura + troca pronominal	Seu namorado fez você vir aqui?
U4→E4	estado (*deprimida*)	molde de empatia	Sinto muito... deprimida.
U5→E5	*infeliz*	molde sobre ajuda	... ajudar a não ser infeliz?
U6→E6	*preciso/ajuda*	pergunta hipotética	O que significaria... ajuda?
U7→E7	`mãe`→família	generaliza o tópico	Fale-me mais sobre a sua família.
U8→E8	acusação ao terapeuta	inverte em pergunta	O que faz você pensar...?
U9→E9	exposição (*computador*)	negação de pressuposto	Importa se eu sou um computador?

3.5 Evoluções: PARRY (1972) e AIML / ALICE (1995–2000)

PARRY (Kenneth Colby, Stanford, 1972). O PARRY simula o discurso de uma pessoa com esquizofrenia paranoica [6]. Diferente do ELIZA, mantém um modelo de personalidade com variáveis afetivas contínuas: medo (*fear*), raiva (*anger*) e desconfiança (*mistrust*). A mesma pergunta do interlocutor gera respostas distintas conforme esses níveis. Com medo e raiva baixos, o sistema pode falar de suas obsessões; com níveis altos, fica hostil ou evasivo. Em avaliações com psiquiatras e transcrições cegas, os médicos nem sempre distinguiram o programa de pacientes reais. Em relação ao ELIZA, o avanço principal é o estado interno que altera a seleção das respostas.

AIML e A.L.I.C.E. (Richard Wallace, 1995–2000). A linguagem AIML (*Artificial Intelligence Markup Language*) e o bot A.L.I.C.E. (*Artificial Linguistic Internet Computer Entity*) padronizam a base conversacional em categorias XML [7]. Há normalização recursiva com a tag `<srai>`, contexto com `<that>` e `<topic>`, e indexação em árvore (*Graphmaster*), o que permite dezenas de milhares de regras com busca rápida. A.L.I.C.E. venceu o Prêmio Loebner em mais de uma edição. Em relação ao ELIZA, o ganho principal é escala, modularidade e controle explícito de contexto, ainda sem compreensão semântica profunda.

Tabela 6: Comparativo das três gerações de chatbots clássicos.

Critério	**ELIZA (1966)**	**PARRY (1972)**	**AIML/ALICE (1995)**
Arquitetura	script + padrões	regras + estados afetivos	categorias XML (Graphmaster)
Modelo interno	quase nenhum	variáveis emocionais	base hierárquica de regras
Contexto	pilha de tópicos	filtrado pelo estresse	`<that>`, `<topic>`
Variações	decomposição rígida	heurísticas de intenção	recursão `<srai>`
Domínio	escuta clínica	paranoia simulada	cobertura conversacional ampla

Nessa linha, o ELIZA mostra que padrões sintáticos bastam para a ilusão conversacional; o PARRY acrescenta estado afetivo; o AIML/ALICE transforma o acervo de regras num formalismo modular e indexável.

Referências

[1] RUSSELL, Stuart J.; NORVIG, Peter. *Inteligência Artificial: uma abordagem moderna*. 3. ed. Rio de Janeiro: Elsevier / Campus, 2013.

[2] CARVALHO, André C. P. L. F. et al. *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*. Rio de Janeiro: LTC, 2011.

[3] LUGER, George F. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*. 6. ed. Boston: Pearson, 2009.

[4] TENÓRIO, Antonio Rodrigo L. A.; GOMES, Bruno B. F. *Lista de Exercícios 1: Inteligência Artificial*. Notebook `Lista_IA.ipynb`, UFAL, 2026.

[5] WEIZENBAUM, Joseph. ELIZA: a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, v. 9, n. 1, p. 36–45, 1966.

[6] COLBY, Kenneth M.; WEBER, Sylvia; HILF, Franklin D. Artificial paranoia. *Artificial Intelligence*, v. 2, n. 1, p. 1–25, 1971/1972.

[7] WALLACE, Richard S. The anatomy of A.L.I.C.E. In: EPSTEIN, R.; ROBERTS, G.; BEBER, G. (eds.). *Parsing the Turing Test*. Dordrecht: Springer, 2009. p. 181–210.